

Data Structures and Algorithms

Assignment - II

Name: Lavanis A

Due Date: 31st March 2026

Question

Write a psuedocode for a linear order deletion algorithm of a B-tree with degree t i.e. each node can have atmost $2t - 1$ children but atleast $t - 1$ children. First, search the key and return "not found" if the key is not there. Otherwise, delete the key for all the cases as described in class.

Theory

Before constructing the psuedocode, let us look at the definition of B-tree and some properties of the same:

- A B-tree of degree t is a balanced search tree whose nodes can store multiple values, can have multiple children i.e. more than 2 and all the leaves of the tree are at the same level.
- A B-tree of degree t is a balanced search tree whose nodes can store multiple values, can have multiple children i.e. more than 2 and all the leaves of the tree are at the same level.
- If the degree of the tree is t then it means that, each node must store atleast $t - 1$ keys and can store a maximum of $2t - 1$ keys, where keys denote each number/element in that node.

- Moreover, if a node has k keys, then the same must have exactly $k + 1$ children.
- If a node of the B-tree is: $[x_1, x_2 \cdots x_k]$ where $x_1 < x_2 < \cdots < x_k$ and let its children be: $c_1, c_2 \cdots c_{k+1}$, then each element in c_i must be between x_{i-1}, x_i for all $2 \leq i \leq k$, elements of c_1 are strictly lesser than x_1 and that of c_{k+1} are greater than x_k .

So, while designing an algorithm for deleting a key in a node, one must ensure that after deleting the key, the resultant must satisfy all the above properties of the B-trees, that the B-tree structure is restored.

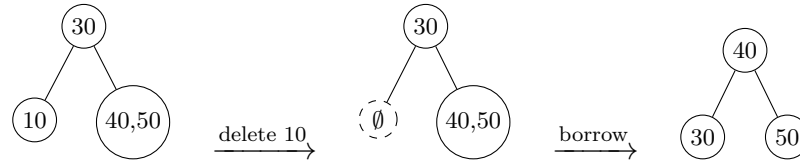
There are 4 scenarios that may arise while deletion and we meticulously resolve the potential errors to revive the B-tree structure after the process:

Case I: Borrowing of a key from sibling:

Suppose after deleting an element from the node, the resultant, node has lesser than $t - 1$ keys and on the other, its sibling has a sufficient number of keys, i.e. more than t keys, then the sibling is eligible to donate one of its keys to keep the B-tree intact.

To execute this, we push a key into from the parent node to the deficient node and to compensate the deficiency in the parent node, an element from the sibling node is pushed to the parent node, thus maintaining the structure.

Look at the following example of a B-tree whose degree is 2.

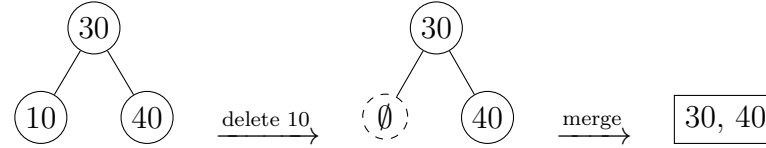


After implementing the steps mentioned above, we have a new B-tree which satisfies all the properties mentioned and the degree of the tree remains unchanged.

Case II: Merge the siblings:

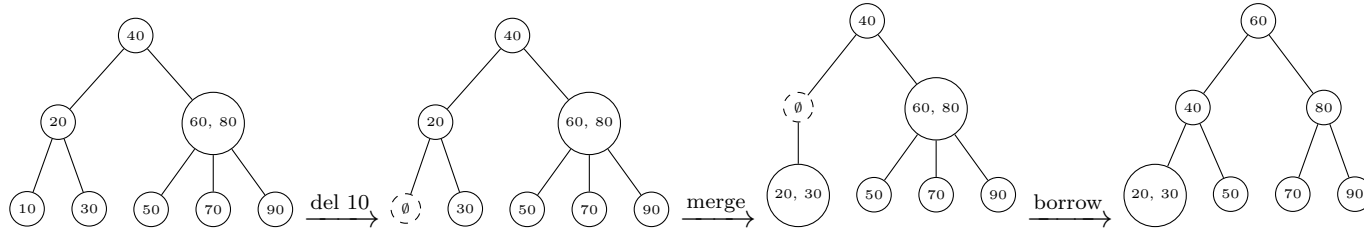
In continuation to Case I, if the sibling nodes have exactly $t - 1$ keys instead of t or more keys, then the

process of donation cannot apply here. Instead, we merge the parent, deficient and the sibling key to form a larger key, let illustrate the same with an example:



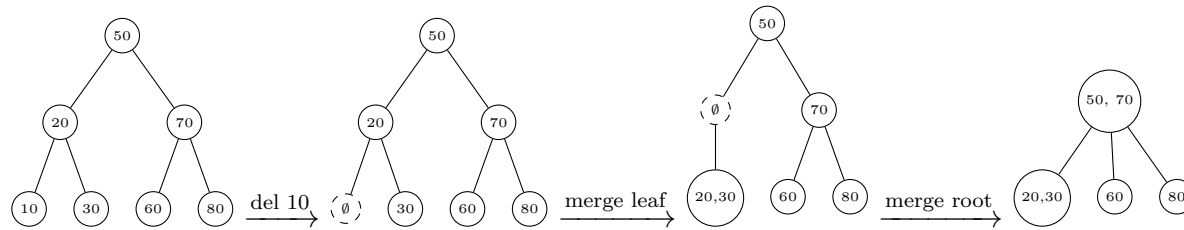
Case III: Merging causes an underflow in Parent node:

If suppose, there's an underflow in the parent node after merging keys, then we perform an action similar to Case I at the parental level. Look at the example below:



Case IV: No plausible borrowing:

Also called the cascading merge, in this case since every possible donor for keys have exactly $t-1$ keys, merging is not possible, instead the tree collapses and the height of it reduces to maintain its B-tree structure. For instance, consider:



Keeping these 4 cases in mind, we now develop the psuedocode for the given problem:

Pseudocode

First things first, we initiate the deletion of the key from the root, and then if the root becomes empty, then we fix the same by calling its child as the new root.

Listing 1: Algorithm 1

```
1 B_TREE_DELETE(T, k):      # T is the tree, k is the key
2   x = T.root              # extract the root node
3   DELETE(x, k)            # recursive delete
4
5   if x.n == 0 and not x.leaf:
6       T.root = x.c[0]
```

where `x.n` denotes the number of keys in the node `x` and `x.c` denotes the set of children of `x`.

We define the predecessor of the key as the largest key in the entire sub-tree (where the node is the root) lesser than the same.

Analogously, we define the successor of the key as the smallest key in the sub-tree which is greater than the same.

We construct the functions that help us determine the successor and predecessor of the keys, which help us in the deletion process:

Listing 2: Algorithm 2 and 3

```
1 GET_PREDECESSOR(x, i): #Finding it for the ith element of node x
2   cur = x.c[i]
3   while not cur.leaf: #Checking it till the very end
4       cur = cur.c[cur.n + 1]
5   return cur.key[cur.n]
6
7 GET_SUCCESSOR(x, i): #Similar logic
8   cur = x.c[i+1]
9   while not cur.leaf:
10       cur = cur.c[1]
11   return cur.key[1]
```

The next code snippet include the psuedocodes for borrowing keys from their siblings and merging nodes, a crucial step in deletion process as one could see in the above mentioned four cases.

Listing 3: Algorithm 4, 5 and 6

```
1 BORROW_FROM_PREV(x, i):
2
3     child = x.c[i]
4     sibling = x.c[i-1]
5
6     shift child keys right
7     child.key[1] = x.key[i-1]
8
9     if not child.leaf:
10         shift children right
11         child.c[1] = sibling.c[sibling.n+1]
12
13     x.key[i-1] = sibling.key[sibling.n]
14
15     child.n += 1
16     sibling.n -= 1
17
18 BORROW_FROM_NEXT(x, i):
19
20     child = x.c[i]
21     sibling = x.c[i+1]
22
23     child.key[child.n+1] = x.key[i]
24
25     if not child.leaf:
26         child.c[child.n+2] = sibling.c[1]
27     x.key[i] = sibling.key[1]
28
29     shift sibling keys left
30     shift sibling children left
31
32     child.n += 1
33     sibling.n -= 1
```

```

34
35 MERGE(x, i):
36
37     child = x.c[i]
38     sibling = x.c[i+1]
39
40     child.key[t] = x.key[i]
41
42     copy sibling keys & children into child
43
44     shift x keys left
45     shift children left
46
47     child.n += sibling.n + 1
48     x.n -= 1

```

In this process, we also ensure that the i th child has atleast t keys. As discussed earlier, the possible ways to restore the deficiency is by borrowing keys from the siblings and if borrowing is not a viable option, then we merge the nodes. In simple terms, we call this step as "FILL" and below is the psuedocode on how to do the same:

Listing 4: Algorithm 7

```

1  FILL(x, i):
2      #Handling Case I
3      if i > 1 and x.c[i-1].n >= t:
4          BORROW_FROM_PREV(x, i)
5
6      else if i <= x.n and x.c[i+1].n >= t:
7          BORROW_FROM_NEXT(x, i)
8      #Handling Case II
9      else:
10         if i <= x.n:
11             MERGE(x, i)
12         else:
13             MERGE(x, i-1)

```

After defining all the functions, we now advance in constructing the psuedocode to delete the key. As discussed in class, the `FIND_KEY(x, k)` function returns the minimum value of i where $x.keys[i] \geq k$, helping us locate key, k .

Listing 5: Algorithm 8

```

1 DELETE(x, k):
2
3     i = FIND_KEY(x, k)
4
5     // Case 1: Key found in node x
6     if i <= x.n and x.key[i] == k:
7
8         // Case 1A: x is a leaf
9         if x.leaf:
10             REMOVE_FROM_LEAF(x, i)
11
12         // Case 1B: x is an internal node
13         else:
14             DELETE_INTERNAL(x, k, i)
15
16     // Case 2: Key not found in this node
17     else:
18         if x.leaf: #Means the key is not in the tree.
19             return "not found"
20
21         flag = (i == x.n + 1)
22
23         // Ensure child has at least t keys
24         if x.c[i].n < t:
25             FILL(x, i)
26
27         // Recurse
28         if flag and i > x.n:
29             DELETE(x.c[i-1], k)
30         else:
31             DELETE(x.c[i], k)
32
33 DELETE_INTERNAL(x, k, i):

```

```

34
35 // Case 2A: Predecessor child has more than t keys
36 if x.c[i].n >= t:
37     pred = GET_PREDECESSOR(x, i)
38     x.key[i] = pred
39     DELETE(x.c[i], pred)
40
41 // Case 2B: Successor child has more than t keys
42 else if x.c[i+1].n >= t:
43     succ = GET_SUCCESSOR(x, i)
44     x.key[i] = succ
45     DELETE(x.c[i+1], succ)
46
47 // Case 2C: Both children have t-1 keys -> merge
48 else:
49     MERGE(x, i)
50     DELETE(x.c[i], k)

```

It was clear that cases I and II are satisfied in the FILL function, but in case II, there's a chance of underflow in parent, the underflow issue is now resolved by the same FILL function and the recursive calls of the DELETE function.

In each node, the following operations are performed:

- Searching for the position of the key using FIND_KEY, which takes $O(t)$ time.
- Possible borrowing or merging operations, which also take $O(t)$ time.

Hence, the total work done at each node is $O(t)$.

The height h of a B-tree with n keys satisfies:

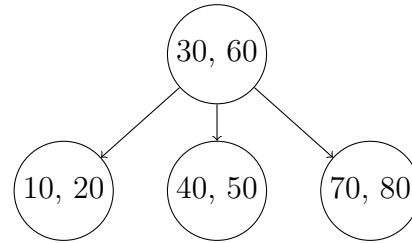
$$h = O(\log_t n)$$

Since the algorithm traverses at most one path from the root to a leaf, the total time is: $O(t \cdot \log_t n)$, the required time complexity.

Execution of algorithm with an example

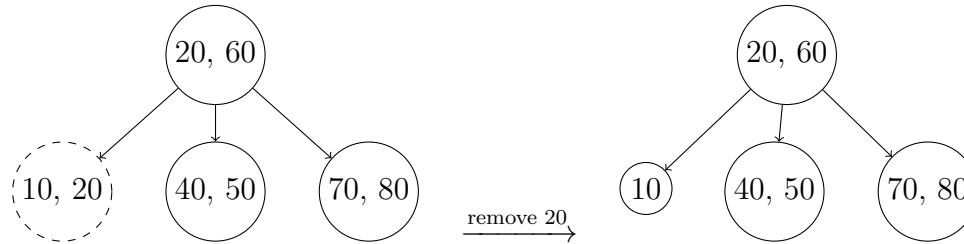
Step 1: Locate Key and Predecessor

We identify key **30**. Since its left child has t keys $\{10, 20\}$, we find the predecessor **20** to replace it.



Step 2: Promotion and Deletion

The key **20** is moved to the root. The original position of the predecessor in the leaf is now empty (or logically removed).



which is the final result.

The flow is as follows:

DELETE(root, 30) $\rightarrow$ FIND_KEY $\rightarrow$ 1 $\rightarrow$ GET_PREDECESSOR $\rightarrow$ 20 $\rightarrow$ Replace key $\rightarrow$ DELETE(left child, 20) $\rightarrow$ REMOVE_FROM_LEAF $\rightarrow$ No FILL required